The Essential of ML: Chapter 1

What it means to learn, cut down to what the exercise sheet asks: rules against experience, what counts as experience, the supervised setup, the four ingredients, generalization and shortcuts

Fundamentals of Machine Learning

2026-10-30

How to read this document

This is Chapter 1 with everything removed that the professor's exercise sheet and Tutorial 1 do not touch. Six sections in concept order: every term is defined before it is used, so read it from the top. Each section ends in a *Tutorial drill* box with questions in the professor's style (explain why, derive, give a counterexample, compute) and the answer written the way an expert writes it: claim, mechanism, a number or a derivation, consequence. Sources are named ("Exercises ch. 1, Q4"; "Tutorial 1, Q1(e)"). Left out on purpose: expert-system history beyond MYCIN, structured outputs (detection, segmentation), the long bigram walkthrough (kept as one computation); the information-theory page is short because the deck gives it one page and the exercise sheet never asks it.

1 Learning, and the Two Routes to a Machine Ability

Learning, capacity, experience

Learning is the process through which **experience** changes a learner and develops what it can do. Experience alone is not enough (a stone exposed to piano lessons learns nothing): the learner also brings a **capacity to learn**, from the brain for animals and from design choices for machines. The chapter's picture is

$$\text{Capacity to Learn} + \text{Experience} \xrightarrow{\text{Learning}} \text{Acquired Abilities}.$$

The abilities we care about fall into four overlapping activities, a vocabulary rather than a taxonomy:

Recognise	**Predict**	**Reason**	**Act**
What is it?	What may happen?	What follows?	What should I do?

How can we give a *machine* such abilities? There are exactly two routes. **Route 1: encode human knowledge.** Humans learn first, then transfer the result as explicit instructions: Human Knowledge → Explicit Rules or Procedures → Computer. Pascal's Pascaline embodied arithmetic in gears; a program embodies a tax law, merge sort, or "hash the password and compare". When a reliable procedure is known this is the right route: learning it again from examples would only add uncertainty. Early AI pushed the route into domains needing expertise: an **expert system** such as MYCIN stored knowledge elicited from specialists as explicit facts and `IF ...THEN ...` rules. As rule bases grew, extracting, expressing and maintaining the knowledge became the hard part: the **knowledge acquisition bottleneck**.

Route 2: make the machine learn. Instead of transferring the *result* of human learning, give the machine experience and a capacity to learn, and let it acquire the ability itself. The question changes from "what rules should humans write?" to "how can we give a machine the capacity to learn from experience?". Designing computational methods that make this possible is *machine learning*. It still involves programming: humans build the learner and its learning mechanism; only the task-specific behaviour is acquired from data instead of written as rules.

Why route 2 exists: the digit "3" thought experiment

You must decide, from a 20×20 binary image, whether it shows a 3. Natural rules: (R1) two bumps opening to the left, i.e. the rightmost black pixel of the top half and of the bottom half both lie in the right third; (R2) the middle row has black pixels only in the right half (the waist); (R3) between 40 and 90 black pixels; (R4) no black pixel touches the left border.
Now perturb the digit. *Shift 4 pixels right*: R1, R4 hold; R2 fails if the waist crosses the centre column. *Write it narrowly*: R1 fails, the bumps no longer reach the right third. *Rotate* 15°: the waist leaves the middle row, R2 fails. *Thick stroke*: the pixel count doubles, R3 fails. Every fix breeds a new failure: "right third" becomes "right third of the bounding box", which a stray dot corrupts; "middle row" becomes "row of minimum width between the bumps", which also matches 5s and 8s; the pixel range widens until it excludes nothing. Rules pile up with thresholds nobody can set and interactions nobody can reason about. **What it reveals**: explicit programming requires us to *know and state* the procedure; learning requires only that we can *recognise correct behaviour on examples*. People recognise a 3 effortlessly and cannot say how.

Key insight. Having an ability $\neq$ having an explicit procedure that describes the ability. The whole field rests on this gap: human vision is a complete ability with no complete description. Where useful experience exists and the procedure is hard to specify, learning is the only route.

Tutorial drill

Q1 (Exercises ch. 1, Q1(d)). At what point would you prefer to give the computer examples of handwritten digits instead of adding rules? What does the thought experiment reveal?
Answer. When writing and maintaining rules costs more than collecting labelled examples: each new batch of handwriting exposes new exceptions, a change to the rule set has unpredictable side effects, and examples are cheap. Mechanism: rules must anticipate every variation (shift, scale, slant, stroke width) explicitly, while examples contain the variations already and the learner absorbs them. The experiment reveals that having the ability (we recognise a 3) does not give us a describable procedure, so route 1 fails exactly on the abilities humans find easiest.
Q2 (Exercises ch. 1, Q2). An expert system for machine faults, built from two hours a week with an engineer, meets a new generation of machines. Separate the difficulties; why would learning help; why does expert knowledge still matter?
Answer. *Obtaining knowledge*: time is scarce and most of the skill is tacit ("that rattle means a loose coupling" cannot be stated as a test), so elicitation is slow and lossy. *Representing it*: judgements are graded ("fairly high"), interacting ("unless just started") and full of exceptions; hundreds of `IF-THEN` rules need fragile thresholds and contradict each other in unanticipated corners. *Unfamiliar situations*: the rules cover only faults the expert knew; on new machines the system is silent or confidently wrong and cannot improve from the failures it observes. Learning attacks all three: patterns come from logged sensor readings and repair records, graded effects are numbers, and the model is retrained on the new machines' data. Expert knowledge still decides *which* measurements to record, sanity-checks the model, and covers rare catastrophic faults with no data.
Q3 (new). Give a counterexample to "if a task is important, we should learn it from data".
Answer. Income tax. The procedure is written in law, so a program applies it exactly; a learned model would approximate the law from past returns and be wrong on rare cases, and here "ap-

proximately correct" is unacceptable. Three questions decide the route: can a competent person write the complete procedure (program it); can we recognise correct behaviour on examples and collect many (learn it); what does a mistake cost (learning is approximately correct; when that is unacceptable, as for safety interlocks or cryptography, keep the program or wrap the learned part in safeguards). Most systems mix both: a learned model predicts, an explicit program decides.

2 What Counts as Experience?

Before any algorithm is chosen we must know what information the learner will receive. Experience reaches a machine in three forms, each leading to a learning setting.

Three forms of experience, three settings

Examples	**Observations**	**Interactions**
input + desired output: direct guidance	observed data only: regularities, nobody labels	actions + consequences: the learner shapes its own future experience
↓ **supervised** learning	↓ **unsupervised** (search for structure) and **self-supervised** (a signal built from the data itself)	↓ **reinforcement** learning (rewards grade behaviour)

These are broad settings, not exclusive kinds of system; one system can use several. **The paradigm is decided by what information about "doing well" reaches the learner, not by the data type**: four projects can all use chest X-rays and belong to four paradigms.

Same hospital, four projects (Exercises ch. 1, Q5)

1. *Predict pneumonia on a new X-ray.* Experience: images paired with diagnoses. Feedback: agreement with the recorded diagnosis. **Supervised.**
2. *Group thousands of unlabelled X-rays by visual similarity.* Experience: images only. Feedback: none from outside, only an internal criterion (within-group similarity), so "unsupervised" is not "no objective". **Unsupervised.**
3. *Hide a region of each X-ray and predict it.* Experience: images only, but each image supplies its own target. Feedback: match with the hidden pixels. **Self-supervised.**
4. *A robot positions the sensor, rewarded for good positions, penalised for unsafe moves.* Experience: its own actions and consequences. Feedback: a delayed, evaluative reward, not a correct answer. **Reinforcement.** Only here does the learner influence its future experience.

Quantity, diversity, representativeness

Three different properties of a collection of experience. **Quantity**: how many examples. **Diversity**: how much of the space of situations they cover. **Representativeness**: whether they are distributed like the situations the deployed system will meet. A learner finds regularities in whatever it is given; a million near-repeats of one situation contain one situation's regularities (this park's foliage, this camera's colour balance), which will not hold elsewhere. Volume does not substitute for coverage.

Tutorial drill

Q1 (Exercises ch. 1, Q3). Ten bird species; Dataset A: one million photographs from one camera in one park; Dataset B: fifty thousand by many people in many countries, seasons, weathers, backgrounds, angles. Which one, and when would the other be preferable?

Answer. For a system used by the public, **B**. Mechanism: A has twenty times the quantity but almost no diversity; its regularities (one park's vegetation, one camera's colour response, probably the same individual birds) are properties of the collection, not of the species, and a learner is free to use them. B covers the variation the deployed system will meet, so its regularities transfer. A is preferable if deployment *is* that park and camera (a fixed camera trap: A is then perfectly representative), or if B's labels are much noisier or B lacks some species.

Q2 (Exercises ch. 1, Q8). Design the experience for a phishing detector; why can a very large dataset still give a poor system?

Answer. Sources: phishing feeds and user reports; legitimate pages from the long tail and the real login pages phishing imitates (the hard negatives). Variety: brands, languages, hosting, kits, mobile and desktop. Correct outcomes: analyst confirmation or agreement of independent blocklists, date recorded, noise expected. Change: phishing turns over in weeks, so collect continuously, test on the future, retrain. A huge dataset from one feed, one kit and one month teaches "phishing = badly spelled page imitating one bank" and is outdated at deployment: badly chosen experience at scale teaches the wrong lesson very confidently.

Q3 (Tutorial 1, Q1(f)). One cybersecurity task per paradigm, with the learning signal.

Answer. Supervised: classify flows benign/attack; signal = the label of each training flow. Self-supervised: predict a masked field of a flow; signal = the hidden part of the traffic, no human label. Reinforcement: an agent chooses block / rate-limit / nothing in a simulated network; signal = a reward (attack contained minus disruption), and its actions change what it sees next.

Q4 (new). Explain why "more data" and "better experience" are not synonyms; give a one-line counterexample.

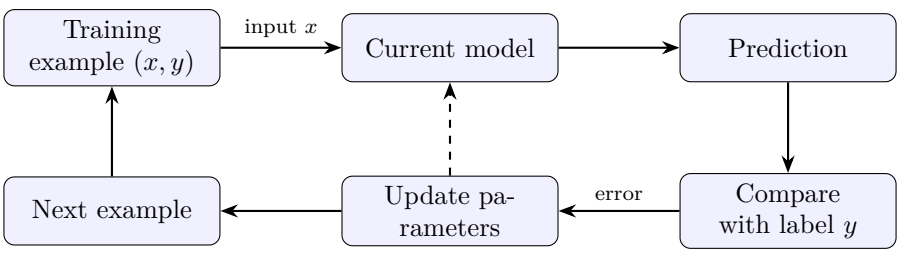
Answer. More data lowers the statistical error of estimating regularities *present* in the collection and does nothing about regularities *absent* from it. A billion photographs of one park are one situation: that park becomes perfectly known, every other place stays unknown. Experience improves with coverage (diversity, representativeness), not count.

3 Supervised Learning: Example, Hypothesis, Loss, Risk

Supervised example, dataset, labels

A **supervised example** is a pair (x, y) of an **input** $x \in \mathcal{X}$ and a **desired output** $y \in \mathcal{Y}$. A **training dataset** is a finite collection $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$. The desired output is called the **target**, the **label** or the **ground truth**; the last term deserves care, because labels come from three imperfect sources: *measurement* (temperature, a recorded sale price: instrument and recording error); *annotation* (an object in an image, a diagnosis read from a scan: experts disagree, and labels depend on conventions fixed when the task was defined); *delayed outcome* (will the loan be repaid, will the pump fail within a month: known only later, hence only for old inputs, in a world that may have changed). Label quality limits what can be learned: with labels wrong 10% of the time, no model can agree with them much more than 90% of the time.

The training cycle. The model receives an input and produces a prediction; the prediction is compared with the desired output; the comparison is feedback the learning algorithm uses to change the model; repeat over the training experience. This process is **training**.



Hypothesis, loss, risk: the vocabulary Chapter 2 makes precise

The **model** is a function $f_\theta : \mathcal{X} \to \mathcal{Y}$ with adjustable **parameters** θ; the set of functions it can represent, $\mathcal{H} = \{f_\theta : \theta \in \Theta\}$, is the **hypothesis space**. A **loss** $\ell(y, f_\theta(x)) \geq 0$ says how bad one prediction is: squared loss $(y - f_\theta(x))^2$ for a number, zero–one loss $\mathbf{1}(y \neq f_\theta(x))$ for a class. The **empirical risk** is the average loss on the training data, computable:

$$\widehat{R}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \ell(y^{(i)}, f_\theta(x^{(i)})).$$

The **population risk** is the average loss over the whole world, $R(\theta) = \mathbb{E}[\ell(Y, f_\theta(X))]$: it is what we want and cannot compute. Training chooses $\hat{\theta}$ by making $\widehat{R}$ small; the hope that $R(\hat{\theta})$ is then small too is the subject of §5.

Regression, classification; prediction, decision, action

A supervised problem is **regression** when y is numerical and **classification** when y is a category from a finite set. The *output* decides, not the input: the same medical image gives regression (tumour size in mm) or classification (benign/malignant); the same student records give classification (will graduate?), regression (final grade) or, with no target at all, an unsupervised grouping (Exercises ch. 1, Q6).

A **prediction** is what the model says about the unknown output; a **decision** uses it together with a *criterion* (costs, a protocol) to select a response; an **action** is what is carried out. "12% probability of disease" (prediction); the protocol recommends testing above 10% (decision); the test is scheduled (action). Change the threshold to 20% and the same prediction yields a different decision; they coincide only when all errors cost the same.

Tutorial drill

Q1 (Tutorial 1, Q1(a)). Network flows labelled benign/attack by incident metadata; the system scores a new flow and decides whether to alert. Identify the physical object, the representation x, the target Y, the model output and the final action.

Answer. Physical object: the completed flow, the actual packet exchange between two endpoints. Representation x: the feature vector computed when the flow completes (duration, packets and bytes per direction, protocol, destination port, TCP flags). Target $Y \in \{0 = \text{benign}, 1 = \text{attack}\}$ from the ground-truthing process. Model output: a score or a posterior estimate $\hat{p}_\theta(y = 1 \mid x)$. Action: alert or not (possibly a third action, send to an analyst). Five distinct objects; merging any two loses the point.

Q2 (Tutorial 1, Q1(b)). Some attack and benign flows have the same observed features. Explain why a classifier on these features cannot separate them perfectly. Are the labels wrong?

Answer. A classifier is a function $h(x)$: one answer per feature vector. If an attack flow and a benign flow share x, h labels both the same and is wrong on one of them, whatever h is; at such an x the error is at least $\min\{\mathbb{P}(Y{=}1 \mid x), \mathbb{P}(Y{=}0 \mid x)\} > 0$. The labels need not be wrong: "attack" is defined by episode membership, so an ordinary-looking reconnaissance request is correctly labelled attack. What is missing is information in x (the host's other flows, the time context), not correctness in y.

Q3 (Tutorial 1, Q1(c)). Why are $\hat{p}_\theta(y = 1 \mid x)$ and the alert decision not the same object?

Answer. The posterior is a *prediction*, a number summarising what the features say; the alert is a *decision*, a rule applied to that number which also needs the costs of a false alarm and of a miss. The same 0.3 should alert where a miss is far costlier than a false alarm and not where alert fatigue dominates; separating them lets you change the threshold without retraining.

Q4 (new, compute). A model predicts prices $210, 180, 300$ for houses sold at $200, 200, 290$ (thousands). Empirical risk under squared loss and under absolute loss? Which one has the

units of money?
Answer. Errors $10, -20, 10$. Squared: $(100 + 400 + 100)/3 = 200$ (thousands)^2; absolute: $(10 + 20 + 10)/3 = 13.3$ thousand. Only the absolute loss is in money; squared loss weights the 20 error four times the 10 errors. Both are $\widehat{R}$, averages over the three houses we have, and say nothing about the next house: that is R.

4 Representations and the Four Ingredients

The form of experience says what information *exists*; the **representation** says what the model *gets*: a learner never sees a house, it sees numbers.

Feature engineering, representation learning, self-supervision

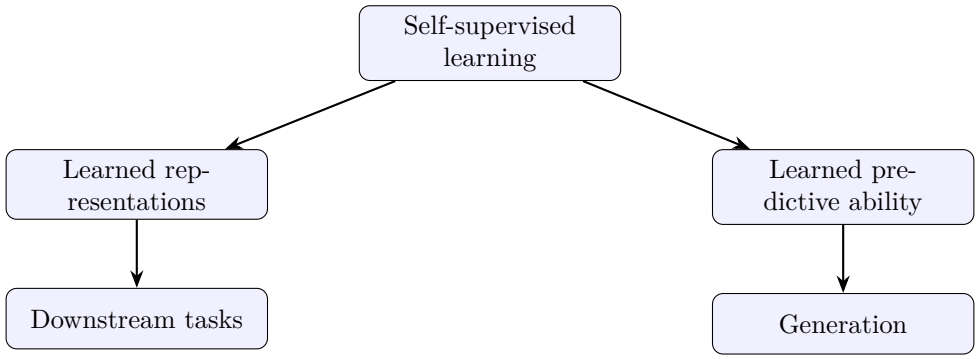
Feature engineering: humans decide which features to present (floor area, location, age for a house; edges, corners, textures for vision; mel-frequency cepstral coefficients for speech) and the model learns how to use them. Perceptual inputs (millions of pixels, a waveform, relationships among words) make the choice hard, and it can decide whether learning succeeds. **Representation learning** lets the machine discover the transformation itself; the idea is older than deep learning, which suits it because successive layers build higher-level features from lower-level ones:

$$\text{Raw Input} \longrightarrow \text{Human-Designed Features} \longrightarrow \text{Learned Predictor},$$
$$\text{Raw Input} \longrightarrow \text{Learned Representation} \longrightarrow \text{Learned Predictor}.$$

Humans still choose architecture, method, experience and objective in the second. **Self-supervised learning** builds a learning problem from observations alone: part of an observation predicts another part (a hidden word from its sentence, a masked image region from the rest, half a second of audio from its context). Nobody labels anything; the data are their own supervision.

Next-token prediction by counting

Corpus: `the cat sat on the mat . the cat ate the rat .` The simplest next-token model looks at the previous word and counts. After `the`: `cat` twice, `mat` once, `rat` once, so $\mathbb{P}(\texttt{cat} \mid \texttt{the}) = \frac{2}{4}$, $\mathbb{P}(\texttt{mat} \mid \texttt{the}) = \mathbb{P}(\texttt{rat} \mid \texttt{the}) = \frac{1}{4}$; after `cat`: `sat`, `ate` each $\frac{1}{2}$; after `sat`: `on` with probability 1. The labels came free from the text. *Generation*: start at `the`, sample `cat`, sample `ate`, then `the` is forced, ...: `the cat ate the mat .`, a sentence absent from the corpus. A large language model is this with thousands of context tokens and billions of parameters in place of a count table.



The two branches are two *uses* of what was learned, not two kinds of model: a generative language model also learns representations.

The four ingredients of a learning system

Experience	What information is available?
Model	In what form can acquired knowledge be represented (and output)?
Learning algorithm	How does experience change the model?
Evaluation criterion	How is success judged?

This answers "what gives a machine the capacity to learn?": the model fixes what can be represented, the algorithm is how experience changes it, the criterion says which changes are useful; the experience supplies the evidence. Representation learning adds that the internal representation itself can be shaped by experience.

Tutorial drill

Q1 (Exercises ch. 1, Q9). Describe an application not used in the chapter without naming an algorithm; map each part to an ingredient.
Answer (hourly electricity demand for a city, next day). (a) Experience: years of hourly demand with weather, calendar and events. (b) Output: a number for each of the 24 hours, possibly an upper quantile since under-supply costs more. (c) Improvement signal: the gap between predicted and realised demand on past days. (d) Success: error on *later* days not used for training, weighted by the asymmetric cost, against the baseline "same as last week". Mapping: (a) experience, (b) model, (c) learning algorithm, (d) evaluation criterion. Design in the order (d), (a), (b), (c): the algorithm has the most textbooks and the least influence on whether the project makes sense.
Q2 (new, compute). In the corpus above, what is the probability that the model, started at `the`, generates exactly `the cat sat on the mat .`? Why is the answer not 1 even though the sentence is in the corpus?
Answer. Multiply the transitions: $\mathbb{P}(\texttt{cat} \mid \texttt{the}) \cdot \mathbb{P}(\texttt{sat} \mid \texttt{cat}) \cdot \mathbb{P}(\texttt{on} \mid \texttt{sat}) \cdot \mathbb{P}(\texttt{the} \mid \texttt{on}) \cdot \mathbb{P}(\texttt{mat} \mid \texttt{the}) \cdot \mathbb{P}(\texttt{.} \mid \texttt{mat}) = \frac{1}{2} \cdot \frac{1}{2} \cdot 1 \cdot 1 \cdot \frac{1}{4} \cdot 1 = \frac{1}{16}$. Not 1 because the model learned *regularities* (what follows what), not sentences: it recombines them, and the unseen `the cat ate the mat .` gets the same $\frac{1}{16}$.

5 Generalization and Shortcuts

Generalization

Experience is always finite (no digit recogniser sees every handwriting, no medical system every patient) and the deployed system meets *new* inputs. **Generalization** is the ability of a learned system to perform well in new situations that were not encountered during learning. The central challenge of machine learning is

$$\text{Finite Experience} \xrightarrow{\text{Learning}} \text{Ability Useful in New Situations.}$$

In the vocabulary of §3: training makes $\widehat{R}(\hat{\theta})$ small; generalization is $R(\hat{\theta})$ being small too.

Why the training data never pin down the rule

Proposition. Let $\mathcal{X}$ be infinite, $\mathcal{Y}$ have at least two elements, and $\mathcal{D}$ be any finite dataset of N pairs. Then infinitely many functions $f : \mathcal{X} \to \mathcal{Y}$ agree with the data, and two of them may disagree on *every* input outside it.
Proof. Let $S = \{x^{(1)}, \ldots, x^{(N)}\}$; $\mathcal{X} \setminus S$ is infinite. Fix $y_0 \neq y_1$ in $\mathcal{Y}$. For each subset $T \subseteq \mathcal{X} \setminus S$ define $f_T(x) = y^{(i)}$ on $x = x^{(i)}$, $f_T(x) = y_1$ on T, $f_T(x) = y_0$ elsewhere. Every f_T fits the data; $T \neq T'$ give different functions; an infinite set has infinitely many subsets. $T = \mathcal{X} \setminus S$ and $T' = \emptyset$

disagree everywhere off the data. □
Consequence. Off the training inputs the data say nothing. Whatever the system does there comes from its *model* and *learning algorithm*, that is, from the designer's assumptions. Generalization is a property of the match between those assumptions and the world, never of the data alone.

The perfect memoriser. "Look x up in the training set and return its label; otherwise return class 0." Training error $\widehat{R} = 0$. On a continuous input space a new x is never in the table, so the rule always answers 0; with two balanced classes $R = \frac{1}{2}$, a coin toss. Zero training error is compatible with having learned nothing.

Shortcuts: right answers for the wrong reasons (Exercises ch. 1, Q4)

A wolf-versus-dog system performs extremely well for its developers and badly after deployment; most training wolves stood in snow, most dogs did not.

1. *Initial success.* "White background ⇒ wolf" agrees with almost all training examples (the proposition guarantees such alternative rules exist) and is far easier to detect than wolf anatomy, so the learner adopted it; the developers' held-out images shared the correlation, so it scored well there too.
2. *Failure.* Users photograph dogs in snow and wolves in summer. The correlation was a property of the *collection*, not of wolves, so the rule is now systematically wrong.
3. *Diagnosis.* Two photographs: a husky in snow and a wolf on green grass. A system that learned wolves gets both right; a system that learned snow gets both wrong.

Nothing here opens the model: the reasoning uses only the definition of generalization, good on seen situations and poor on new ones because a regularity held in the experience and not in the world.

Shortcut versus target leakage

Target leakage: a feature that would *not be available at decision time*, or that is a consequence of the label (the incident ticket ID, "host quarantined afterwards", a blocklist entry). It predicts the label almost perfectly in the dataset and does not exist for a new input. Fix: remove the field. **Shortcut**: a feature that *is* available at decision time but whose association with the label is an artefact of how the data were collected (snow, time of day, hospital tag). The feature means something else at deployment. Fix: collect data where the association is broken (attacks by day, dogs in snow) and test on them; dropping the feature is one option, not the only one. Testing without opening the model: *hold out* unseen examples, *break the shortcuts* (decouple the cue from the label), *shift the conditions* (another camera, hospital, year), and list the situations absent from the experience, where behaviour is an assumption, not a learned fact.

Tutorial drill

Q1 (Tutorial 1, Q1(e)). Most attacks in the data occurred at night, most benign flows by day. How does time of day become a shortcut, and why is this different from target leakage?
Answer. In the collected data "hour of day" predicts the label well, so the model can use it instead of learning what attack traffic looks like; at deployment a daytime attack is missed and night-time backups raise alerts. It is a shortcut, not leakage: the hour *is* available when a new flow is scored and is not caused by the label; the association was created by the collection (a red-team campaign run overnight) and is not stable. Leakage is fixed by removing the field; a shortcut by collecting data in which the nuisance factor varies and testing on them.
Q2 (Exercises ch. 1, Q7). A warehouse robot trained only in a simulator with flat floors, identical friction and constant lighting fails in reality. You may change the simulator, not the algorithm. What do you vary? Would more visual realism fix it?
Answer. Vary what varies in reality and affects the decisions: floor slope and bumps, box mass

and friction, lighting, sensor noise, wheel slippage, worker behaviour. A policy trained across the range cannot rely on any one constant (it never knows the exact friction), so it must work across conditions; the real warehouse becomes one more variation already handled. Photorealism alone does not help: the mismatch is largely *physical*; a photorealistic simulator with flat floors still teaches that wheels never slip.

Q3 (Exercises ch. 1, Q10). "Our AI has learned every possible situation from millions of examples." What is wrong, without mathematics?

Answer. The claim confuses situations that were *observed* with situations that *may occur*. Millions of examples are a finite sample, often near-repeats, so their number says little about how much of the world they cover; the deployed system meets new people, places and conditions, and the world changes after collection (in security, an adversary deliberately does the unseen). The honest claim is good performance on what was seen and an *expectation* that the regularities carry over; whether they do is generalization, testable only on new situations. Had every situation been seen, a lookup would do.

Q4 (new, compute). Removing "hour of day" from a flow classifier raises validation error from 0.04 to 0.21. On a later week with attacks in office hours, the full model scores 0.35, the model without the hour 0.08. Interpret; why could the ablation alone not have raised the alarm?

Answer. The ablation says the model leans on the hour five-fold more than on any flow property: suspicious (the hour describes the capture, not the flow) but not proof. The new week settles it: 0.35 against 0.08, a shortcut costing 0.27 at deployment while gaining 0.17 on the original split. The ablation could not see it because the validation set came from the same collection, where "night ⇒ attack" held on both sides of the split; only data collected under different circumstances separate a stable regularity from an artefact.

6 A Connection to Information Theory

Entropy, cross-entropy, divergence (the deck's last page; Chapter 2 trains with it)

For distributions p and q on C outcomes (with $q_c > 0$ wherever $p_c > 0$, and $0 \log 0 = 0$):

$$H(p) = -\sum_c p_c \log p_c, \qquad H(p,q) = -\sum_c p_c \log q_c, \qquad \mathrm{KL}(p \,\|\, q) = \sum_c p_c \log \frac{p_c}{q_c}.$$

$H(p)$: how uncertain the truth is. $H(p,q)$: the average surprise when you believed q and the truth was drawn from p. Expanding the divergence, $H(p,q) = H(p) + \mathrm{KL}(p\|q)$. Base 2 gives bits, base e nats.

Gibbs' inequality. $\mathrm{KL}(p\|q) \geq 0$ with equality iff $q = p$; hence $H(p,q) \geq H(p)$, with the minimum at $q = p$. *Proof in two lines*: $\ln t \leq t - 1$ for $t > 0$ (equality iff $t = 1$); apply it to $t = q_c/p_c$ and average under p: $-\mathrm{KL} = \sum_c p_c \ln(q_c/p_c) \leq \sum_c (q_c - p_c) = \sum_c q_c - 1 \leq 0$.

Numbers: truth $p = (0.7, 0.3)$, in bits

$H(p) = -0.7 \log_2 0.7 - 0.3 \log_2 0.3 \approx 0.881$, the floor no prediction beats. Prediction $(0.5, 0.5)$: $H(p,q) = 1.000$, $\mathrm{KL} = 0.119$ (hedging costs 0.119 bits). Prediction $(0.9, 0.1)$: $H(p,q) = 0.7 \times 0.152 + 0.3 \times 3.322 = 1.103$: overconfident in the *right* direction, still worse than hedging. Prediction $(0.99, 0.01)$: 2.003 bits; the 0.3 mass pays $\log_2 100 = 6.64$ bits each time. One-hot truth: cross-entropy is $-\log q_y$, the training form of Chapter 2. Lessons: the floor $H(p)$ is set by the problem; overconfidence is punished sharply; by Gibbs, minimising cross-entropy trains *calibrated* probabilities, not just the right argmax. Representation learning in the same language: keep the information about the output, discard the rest.

Tutorial drill

Q1 (Quiz refresher ch. 1–3, A8). Write $H(p)$, $H(p,q)$, $\mathrm{KL}(p\|q)$; state Gibbs' inequality and what it implies about minimising cross-entropy.
Answer. Formulas as in the box. Gibbs: $\mathrm{KL}(p\|q) \geq 0$ with equality iff $q = p$, so $H(p,q) = H(p) + \mathrm{KL}(p\|q)$ is minimised exactly at $q = p$, with minimum $H(p)$: minimising expected cross-entropy pushes a classifier towards the true conditional probabilities.
Q2 (new, compute). Truth $(0.5, 0.5)$, prediction $(0.99, 0.01)$: cross-entropy in bits, against the honest prediction? What if the prediction were $(1, 0)$?
Answer. $H(p,q) = -0.5\log_2 0.99 - 0.5\log_2 0.01 = 0.5 \times 0.0145 + 0.5 \times 6.644 \approx 3.33$ bits against 1 bit for $q = p$: overconfidence more than triples the penalty. With $q = (1, 0)$, $H(p,q) = +\infty$: never predict exactly 0 for something that can happen.

The mistakes that cost points

- Naming the paradigm from the data type ("X-rays, so supervised"). The feedback that reaches the learner decides: a label per example, nothing external, a target made from the data, or a reward earned by acting.
- Answering "one million is more than fifty thousand". Generalization depends on diversity and representativeness, not quantity; say which deployment makes which dataset representative.
- Calling a shortcut "leakage". Leakage: unavailable at decision time, fix by removing it. Shortcut: available but spuriously associated, fix by breaking the association in the data and testing on it. And a probability is a prediction, not yet a decision.

Chapter 1 in five lines

1. Learning = capacity to learn + experience → ability. Two routes: write the rules (procedure known, exactness matters) or learn from experience (correct behaviour recognisable, procedure not describable). Ability ≠ procedure; expert systems hit the knowledge acquisition bottleneck.
2. Experience: examples, observations or interactions, giving supervised, unsupervised/self-supervised and reinforcement learning; the feedback decides. Quantity, diversity and representativeness are three different things.
3. Supervised learning: pairs (x, y) with imperfect labels; hypothesis $f_\theta \in \mathcal{H}$, loss ℓ, computable $\widehat{R}$, wanted R. The output type decides regression versus classification; prediction, decision, action are three objects.
4. Representations decide what the model sees, engineered or learned; self-supervision makes targets from the data, next-token prediction sampled forward is generation. Four ingredients: experience, model, learning algorithm, evaluation criterion.
5. Generalization is the goal: finite data fit infinitely many rules, so off-sample behaviour comes from assumptions; zero training error fits a memoriser or a snow detector. Shortcut (available, spurious) ≠ leakage (unavailable at decision time). $H(p,q) = H(p) + \mathrm{KL}(p\|q)$, minimised at $q = p$ (Gibbs).